PataFácil

Roteiro de apresentação do PIM IV

Análise e Desenvolvimento de Sistemas

ALUNO 1	Abertura, arquitetura, Orientação a Objetos, segurança, banco de dados e estoque
ALUNO 2	Regras de negócio, agendamento, balcão e ordem de serviço, Inteligência Artificial, testes e encerramento

Banca: 2 professores · Duração prevista: cerca de 20 minutos + perguntas

1. Como a apresentação está dividida

Cada aluno apresenta uma metade do sistema e conduz a demonstração ao vivo da sua parte. Os tempos são uma sugestão para cerca de 20 minutos; ajustem ao tempo que a banca der. Troquem Aluno 1 e Aluno 2 pelos nomes de vocês.

Ordem	Quem	Assunto	Tempo
1	Aluno 1	Abertura: problema, objetivo e perfis de usuário	2 min
2	Aluno 1	Arquitetura e camadas (Controller, BLL, DAL, Models)	3 min
3	Aluno 1	Orientação a Objetos no código	2 min
4	Aluno 1	Segurança (JWT, perfis, BCrypt) + demonstração de login e erro 403	2 min
5	Aluno 1	Banco de dados e estoque (triggers, entrada de mercadoria, fornecedores) + demonstração	3 min
6	Aluno 2	Regras de negócio e o fluxo de agendamento	2 min
7	Aluno 2	Demonstração ao vivo: tutor agenda, balcão recebe o cliente, dá baixa no pagamento e emite a ordem de serviço	3 min
8	Aluno 2	Inteligência Artificial (3 modelos)	2 min
9	Aluno 2	Testes, DevOps, mobile e acessibilidade	1 min
10	Aluno 2	Limitações, melhorias futuras e encerramento	1 min

Antes de começar (checklist)

- MySQL, API de IA (porta 8000) e API em C# (porta 5288) rodando. Teste http://localhost:5288/health e http://localhost:8000/health.
- Navegador aberto em Frontend/index.html (aperte Ctrl+F5 para carregar a versão mais recente) e uma segunda aba no Swagger (http://localhost:5288/swagger).
- O sistema já cria uma cliente de demonstração com pets e agendamentos: cliente@patafacil.com (Maria Souza, pets Thor e Mel), e um catálogo com 25 serviços, 51 produtos e 6 fornecedores. Usem esses dados para não perder tempo.
- VS Code aberto com os arquivos que serão mostrados: Program.cs, AgendamentoService.cs, Agendamento.cs, Pessoa.cs, AuthController.cs, EstoqueService.cs, ScriptBancoDeDados.sql e ml_api.py.
- Usuários de teste, todos com senha 123456: gerente@patafacil.com, atendente@patafacil.com (Ana, abre direto na agenda), vet@patafacil.com, carla.tosa@patafacil.com (Banho e Tosa) e cliente@patafacil.com.
- Combinar quem opera o computador: quem está falando é quem mexe na tela.

Dicas

- Ensaiem pelo menos duas vezes juntos, cronometrando.
- Se algo der erro na demonstração, mantenham a calma, expliquem o que deveria acontecer e mostrem o código da regra.
- Falem olhando para a banca, não para a tela.
- Não decorem palavra por palavra: entendam a ideia de cada bloco e falem com as suas palavras.

2. Roteiro do Aluno 1

ALUNO 1 **Abertura, arquitetura, OO, segurança, banco e estoque** **≈ 12 min**

2.1 Abertura

O QUE FALAR

*"Bom dia, professores. Nosso projeto é o **PataFácil**, um sistema de gestão para pet shop e clínica veterinária. O problema que resolvemos é a desorganização da operação: horários marcados em duplicidade, clientes que faltam sem avisar, estoque controlado no papel e nenhum comprovante do atendimento. O sistema cuida de **agendamentos, balcão de atendimento, ordem de serviço, prontuário clínico, estoque, compras e vendas, relatórios** e tem uma parte de **Inteligência Artificial** que prevê faltas, demanda e o perfil dos clientes. Há quatro perfis: o **Tutor**, que é o dono do pet, e a equipe, formada por **Atendente, Veterinário e Gerente**, além dos profissionais de banho e tosa. Eu vou apresentar a estrutura técnica do sistema e o meu colega apresenta as regras de negócio, o balcão e a IA."*

MOSTRAR NA TELA

Tela de login do sistema (Frontend/index.html).

2.2 Arquitetura

O QUE FALAR

*"O sistema tem quatro partes. O **frontend web** em HTML, CSS e JavaScript, e um **aplicativo mobile** em React Native com Expo, para o tutor. As duas interfaces conversam com a mesma **API em C# ASP.NET Core**, que é a única que acessa o **banco MySQL**. A API também chama um **microsserviço de IA em Python** com FastAPI. Ou seja: nenhuma tela acessa o banco diretamente. Tudo passa pela API, que aplica as regras e a segurança."*

MOSTRAR NA TELA

Desenhar ou mostrar o esquema:

```
Frontend / Mobile -> API C# -> MySQL
API C# -> API de IA (Python)
```

2.3 Camadas do backend

O QUE FALAR

*"Dentro da API usamos **arquitetura em camadas**. Os **Controllers** recebem as requisições HTTP e verificam quem pode chamar cada rota. A **BLL**, a camada de negócio, tem as regras, por exemplo o AgendamentoService e o EstoqueService. A **DAL** faz o acesso ao banco com o **padrão Repository** e o Entity Framework. E os **Models** são as classes que representam as tabelas. Cada camada só conhece a de baixo, então dá para mudar uma sem quebrar as outras. Tudo é ligado por **injeção de dependência**, configurada no Program.cs."*

MOSTRAR NA TELA

VS Code: as pastas Controllers, BLL, DAL e Models; depois **Program.cs**, linhas 31 a 40 (registro dos serviços e do Repositorio genérico).

2.4 Orientação a Objetos

O QUE FALAR

*"Aplicamos os quatro pilares. **Abstração e herança**: a classe Pessoa é abstrata, guarda nome, CPF e e-mail, e Tutor e Funcionario herdam dela. **Polimorfismo**: Pessoa declara o método GetPerfil e cada filha responde de um jeito; no login tratamos tutor e funcionário como Pessoa. **Encapsulamento**: na classe Agendamento o Status tem private set. Ninguém altera o status diretamente; é preciso chamar Confirmar, Cancelar, Concluir, RegistrarChegada ou RegistrarPagamento, e cada método valida se a mudança é permitida. Por fim, o Repositorio genérico funciona para qualquer entidade, sem repetir código."*

MOSTRAR NA TELA

Models/Pessoa.cs (classe abstrata) e Models/Agendamento.cs, linha 29 (private set) e métodos Confirmar, Cancelar, Concluir, RegistrarChegada (linha 66) e RegistrarPagamento (linha 82).

2.5 Segurança + demonstração

O QUE FALAR

*"O acesso é feito com **token JWT**. Ao fazer login, a API gera um token assinado que vale 8 horas e carrega o id e o perfil do usuário. Cada rota diz quais perfis podem usá-la; um tutor que tenta acessar uma tela de gerente recebe **erro 403**. O tutor também só enxerga os próprios pets e agendamentos. As senhas são guardadas com hash **BCrypt**, nunca em texto puro. E a mensagem de erro do login é a mesma para e-mail inexistente e senha errada, para não revelar quem tem cadastro."*

MOSTRAR NA TELA

1) Login como gerente@patafacil.com e mostrar a Visão Geral. 2) No Swagger, chamar uma rota gerencial sem token ou com token de tutor e mostrar o 401/403. 3) Mostrar **AuthController.cs**, linhas 56 e 74 (mensagem única e expiração de 8 horas).

2.6 Banco de dados e estoque + demonstração

O QUE FALAR

*"O banco é MySQL. O script ScriptBancoDeDados.sql é aplicado sozinho quando a API sobe e cria **15 tabelas**, além de triggers, procedures, views e índices; pode ser executado de novo sem apagar dados. Um exemplo: a trigger **trg_item_venda_baixa_estoque** diminui o estoque a cada item vendido, que é a **RN15**. Colocamos isso no banco para a baixa acontecer mesmo se alguém inserir uma venda por fora da API. Quando o estoque chega no ponto de reposição, aparece um alerta para o gerente e o atendente. Na outra ponta, a **entrada de mercadoria** registra a nota fiscal, a validade do lote, o custo e o preço; o sistema sugere o preço pela margem de lucro sobre a venda, e o banco recusa preço abaixo do custo. O produto pode ser cadastrado na própria entrada, e o **fornecedor** é escolhido entre os cadastrados, já vem preenchido com o último fornecedor daquele produto, ou é cadastrado na hora."*

MOSTRAR NA TELA

1) estoque.html: mostrar os alertas de reposição e de validade no topo. 2) Fazer uma venda e mostrar o estoque diminuindo. 3) Aba Entrada de Mercadoria: escolher um produto, mostrar o fornecedor preenchido e o cálculo da margem; clicar em **+ Novo fornecedor** e mostrar a janela de cadastro. 4) No VS Code, mostrar a trigger no ScriptBancoDeDados.sql (linha 351) e a tabela fornecedores (linha 266).

Passagem para o Aluno 2: "Essa é a base técnica do sistema. Agora o meu colega vai mostrar como as regras de negócio funcionam na prática, o balcão e a parte de Inteligência Artificial."

3. Roteiro do Aluno 2

ALUNO 2 **Regras de negócio, balcão, IA, testes e encerramento** **≈ 9 min**

3.1 Regras de negócio

O QUE FALAR

"O coração do sistema é o agendamento, e ele segue regras que o sistema não deixa quebrar. ***RN01****: nunca dois serviços com o mesmo profissional no mesmo horário.* ***RN02****: o tutor só cancela pela internet com pelo menos 2 horas de antecedência; a equipe pode cancelar a qualquer momento.* ***RN03****: atendimento das 8h às 19h, fechado aos domingos.* ***RN04****: o status segue Pendente, depois Confirmado, depois Concluído, sem pular nem voltar etapa.* ***RN10****: o usuário escolhe só o início; o fim é calculado somando a duração do serviço."*

MOSTRAR NA TELA

Tabela de regras de negócio do relatório ou o GUIA_DE_EXECUCAO.txt, seção 5.

3.2 O fluxo de criação de um agendamento

O QUE FALAR

"Quando um agendamento chega, o AgendamentoService faz nesta ordem: busca o serviço para saber a duração; recusa datas no passado; calcula o horário de fim; recusa domingo e horários fora do expediente; se quem pede é um tutor, confere se o pet é dele; e verifica conflito de horário. Para o conflito usamos a fórmula clássica de ***sobreposição de intervalos****: início do novo menor que o fim do existente, e fim do novo maior que o início do existente. Se passar em tudo, salva, consulta a IA e envia e-mail de confirmação. A IA e o e-mail ficam dentro de try/catch: se a IA estiver fora do ar, o agendamento é feito mesmo assim. A IA é um apoio, não um ponto de falha."*

MOSTRAR NA TELA

BLL/AgendamentoService.cs: linhas 102 e 103 (RN10), linhas 105 a 115 (RN03), linhas 124 a 136 (RN01, a fórmula), linhas 145 a 181 (chamada à IA com tempo limite de 3 segundos).

3.3 Demonstração ao vivo: do agendamento à ordem de serviço

O QUE FALAR

*“Vou mostrar como cliente. Escolho o pet e um serviço de banho e tosa; o sistema lista os três profissionais de banho e tosa e só os horários livres, de 15 em 15 minutos. Confirmo, e o cliente recebe o e-mail de confirmação. Agora entro como a atendente Ana: o sistema abre direto na agenda. Navego pelo dia ou pela semana e vejo o pet, o cliente e o telefone com WhatsApp, e posso anotar observações sobre o pet. Quando o cliente chega, clico em **Cliente chegou** e ele vai para a aba Em processo. Quando o pet fica pronto, clico em **Finalizar e receber**, informo o valor e a forma de pagamento, e o atendimento é concluído. Se eu tentar receber antes de registrar a chegada, o sistema bloqueia, porque essa regra está dentro da própria classe Agendamento. Todo atendimento tem a sua **ordem de serviço**, gerada automaticamente: posso imprimir em A4, baixar em PDF para mandar ao cliente ou imprimir o cupom numa impressora térmica de 80 ou 58 milímetros.”*

MOSTRAR NA TELA

1) Login como cliente@patafacil.com e agendar em agendamento.html. 2) Mostrar a Caixa de E-mail do cliente com a confirmação. 3) Login como atendente@patafacil.com: agenda abre sozinha; mostrar Dia/Semana e as setas. 4) Escrever uma observação, clicar em Cliente chegou, ir em Em processo e Finalizar e receber (escolher Dinheiro e mostrar o troco). 5) No mesmo cartão, clicar em **Ordem de serviço**: mostrar A4, trocar para Térmica 80 mm e clicar em Baixar PDF. 6) Mostrar a tela Ordens de Serviço com o filtro por situação.

3.4 Inteligência Artificial

O QUE FALAR

*“A IA é um microsserviço em Python com FastAPI e scikit-learn, com três modelos. O primeiro é um **Random Forest** que prevê o risco de o cliente faltar, usando a antecedência, os cancelamentos anteriores, o serviço, o dia da semana e se o tutor é novo; ele tem **74% de acurácia**. O segundo é uma **Regressão Linear** que prevê o movimento do dia, com **R² de 0,83**, e aparece na Visão Geral. O terceiro é o **K-Means**, que agrupa os clientes em quatro perfis: Ocasionais, Em Risco, Fiéis e VIP. Como o pet shop ainda não tem histórico real suficiente, os modelos são treinados com **dados sintéticos** que seguem o comportamento esperado do negócio. Quando houver dados reais, basta trocar as funções que geram os dados pela leitura do banco.”*

MOSTRAR NA TELA

1) http://localhost:8000/health mostrando os 3 modelos e as métricas. 2) dashboard.html com a demanda prevista e o gráfico de movimento da semana. 3) MachineLearning/ml_api.py: as funções gerar_dados_* e o treino dos 3 modelos.

3.5 Testes, DevOps, mobile e acessibilidade

O QUE FALAR

*“Temos **28 testes automatizados** com xUnit e Moq que cobrem as regras RN01, RN02, RN03, RN04, RN10 e RN15, a chegada e o pagamento no balcão, a entrada de mercadoria com o cálculo da margem, além do BCrypt. Eles rodam sem banco, porque os repositórios são simulados; isso só é possível graças à injeção de dependência. Temos também Dockerfiles, docker-compose e um pipeline de CI no GitHub Actions. O tutor tem um aplicativo mobile em React Native que usa a mesma API. E todas as páginas têm o **VLibras**, que traduz o conteúdo para Libras, com uma interface padronizada e de leitura confortável.”*

MOSTRAR NA TELA

Pasta Tests/ (AgendamentoServiceTests.cs e EstoqueServiceTests.cs), o resultado do dotnet test e o ícone do VLibras em qualquer tela.

3.6 Limitações e encerramento

O QUE FALAR

“Como melhorias futuras: treinar a IA com dados reais do banco; proteger a verificação de conflito também com uma trava no banco, para o caso de duas pessoas agendarem no mesmo segundo; ligar o envio de e-mails a um serviço real; e, em produção, guardar a chave do JWT numa variável de ambiente e restringir o CORS. Concluindo: o PataFácil organiza a agenda, impede erros de marcação, acompanha o cliente do balcão até a ordem de serviço, controla compras e estoque automaticamente e usa IA para antecipar faltas e demanda. Obrigado. Estamos à disposição para perguntas.”

MOSTRAR NA TELA

Voltar para a Visão Geral ou para a tela de login.

4. Perguntas prováveis da banca

Quem é indicado na coluna “Responde” começa a resposta; o outro aluno pode complementar. Admitir uma limitação e explicar como resolvê-la costuma ser mais bem avaliado do que fugir da pergunta.

Pergunta	Responde	Resposta curta
Qual a diferença entre Controller e Service?	Aluno 1	O Controller cuida do HTTP e das permissões; o Service tem as regras de negócio. O Controller só repassa o pedido.
Por que usar o padrão Repository?	Aluno 1	Isola o acesso ao banco: o Service não sabe que é MySQL. Um único Repositorio<T> atende todas as entidades, e nos testes ele é trocado por um simulado.
Onde está o polimorfismo?	Aluno 1	Pessoa.GetPerfil() é abstrato; Tutor e Funcionario implementam. No login os dois são tratados como Pessoa.
Como funciona o JWT?	Aluno 1	No login a API gera um token assinado com uma chave secreta, contendo id e perfil, válido por 8 horas. Cada requisição envia o token e a API confere a assinatura e o perfil.
Por que a baixa de estoque é uma trigger e não código C#?	Aluno 1	Garante a RN15 mesmo se a venda for gravada por outro caminho, e evita baixar duas vezes.
Como o preço de venda é sugerido?	Aluno 1	Pela margem sobre a venda: preço = custo ÷ (1 − margem). Com custo de R$ 100 e margem de 45%, o preço é R$ 181,82, e o lucro de R$ 81,82 é 45% do preço. Somar 45% ao custo daria só 31% de margem.
Por que a entrada guarda o nome do fornecedor e não só o id?	Aluno 1	Para preservar o histórico da nota como ela foi recebida. O cadastro de fornecedores tem nome único, e um nome digitado direto na entrada entra no cadastro automaticamente.
A chave do JWT está no appsettings.json. Isso é seguro?	Aluno 1	Para desenvolvimento, sim. Em produção iria para variável de ambiente ou cofre de segredos, e o CORS seria restrito ao domínio do site.
E se duas pessoas agendarem o mesmo horário no mesmo segundo?	Aluno 2	A checagem da RN01 é feita antes de salvar, então em tese as duas poderiam passar. A melhoria é usar transação ou trava no banco.
Como a ordem de serviço é gerada? Ela tem tabela própria?	Aluno 2	Não precisa: a OS é montada a partir do agendamento e numerada pelo id dele, então sempre existe e nunca fica desatualizada. A impressão usa o tamanho de página do CSS (A4 ou bobina de 80/58 mm) e o PDF é gerado no próprio navegador.
Os dados da IA são reais?	Aluno 2	Não, são sintéticos, porque o pet shop ainda não tem histórico. As regras de geração imitam o negócio; com dados reais, basta trocar as funções gerar_dados_*.
74% de acurácia é bom?	Aluno 2	Para um modelo de apoio que só coloca uma etiqueta e não bloqueia nada, é aceitável. Com dados reais a tendência é melhorar.
Por que a IA está em Python e não em C#?	Aluno 2	O scikit-learn é o padrão de mercado para Machine Learning. Como microsserviço separado, se a IA cair o resto do sistema continua funcionando.
O que acontece se a IA estiver desligada?	Aluno 2	Nada quebra: a chamada tem tempo limite de 3 segundos e fica dentro de try/catch. O agendamento é salvo sem a etiqueta de risco.

Pergunta	Responde	Resposta curta
Como vocês garantem que as regras funcionam?	Aluno 2	Com os 28 testes automatizados (xUnit e Moq), que rodam sem banco e cobrem as principais regras.
Como o sistema impede receber um pagamento duas vezes?	Aluno 2	Chegada e pagamento são métodos da classe Agendamento (encapsulamento). O pagamento exige a chegada registrada e recusa uma segunda baixa.